WEEK 1 AND 2:

For this week's lab task, I built a Python tool using the Biopython library that takes a DNA or RNA sequence as input and processes it through three steps. First, it validates the sequence to make sure it only contains valid nucleotide characters. Then it transcribes the DNA into RNA, and finally translates the RNA into a protein sequence starting from the AUG start codon. **To test whether my tool was actually working correctly, I used the real human insulin gene sequence from NCBI and ran it through my tool. The protein sequence my tool produced was then compared to the official human insulin protein entry on UniProt. The two sequences matched completely, which confirms that my tool is correctly performing transcription and translation and producing biologically accurate results.**

DNA SEQUENCE USED:

AGCCCTCCAGGACAGGCTGCATCAGAAGAGGCCATCAAGCAGATCACTGTCCTTCTGCCATGGCCCTGTG
GATGCGCCTCCTGCCCCTGCTGGCGCTGCTGGCCCTCTGGGGACCTGACCCAGCCGCAGCCTTTGTGAAC
CAACACCTGTGCGGCTCACACCTGGTGGAAGCTCTCTACCTAGTGTGCGGGGAACGAGGCTTCTTCTACA
CACCCAAGACCCGCCGGGAGGCAGAGGACCTGCAGGTGGGGCAGGTGGAGCTGGGCGGGGGCCCTGGTGC
AGGCAGCCTGCAGCCCTTGGCCCTGGAGGGGTCCCTGCAGAAGCGTGGCATTGTGGAACAATGCTGTACC
AGCATCTGCTCCCTCTACCAGCTGGAGAACTACTGCAACTAGACGCAGCCCGCAGGCAGCCCCACACCCG
CCGCCTCCTGCACCGAGAGAGATGGAATAAAGCCCTTGAACCAGC

The output

MALWMRLLPLLALLALWGPDPAAAFVNQHLCGSHLVEALYLVCGERGFFYTPKTRREAEDLQVGQVELGG
GPGAGSLQPLALEGSLQKRGIVEQCCTSICSLYQLENYCN

From UniProt:

MALWMRLLPLLALLALWGPDPAAAFVNQHLCGSHLVEALYLVCGERGFFYTPKTRREAEDLQVGQVELGG
GPGAGSLQPLALEGSLQKRGIVEQCCTSICSLYQLENYCN

```
CODON_TABLE = {
    #Phenylalanine
    'UUU': 'F', 'UUC': 'F',
    #Leucine
    'UUA': 'L', 'UUG': 'L',
    'CUU': 'L', 'CUC': 'L',
    'CUA': 'L', 'CUG': 'L',
    #Serine
    'UCU': 'S', 'UCC': 'S',
```

```
    'UCA': 'S', 'UCG': 'S',
    'AGU': 'S', 'AGC': 'S',
    #Tyrosine
    'UAU': 'Y', 'UAC': 'Y',
    #Cysteine
    'UGU': 'C', 'UGC': 'C',
    #Tryptophan
    'UGG': 'W',
    #Proline
    'CCU': 'P', 'CCC': 'P',
    'CCA': 'P', 'CCG': 'P',
    #Histidine
    'CAU': 'H', 'CAC': 'H',
    #Glutamine
    'CAA': 'Q', 'CAG': 'Q',
    #Arginine
    'CGU': 'R', 'CGC': 'R',
    'CGA': 'R', 'CGG': 'R',
    'AGA': 'R', 'AGG': 'R',
    #Isoleucine
    'AUU': 'I', 'AUC': 'I',
    'AUA': 'I',
    #Methionine
    'AUG': 'M',
    #Threonine
    'ACU': 'T', 'ACC': 'T',
    'ACA': 'T', 'ACG': 'T',
    #Asparagine
    'AAU': 'N', 'AAC': 'N',
    #Lysine
    'AAA': 'K', 'AAG': 'K',
    #Valine
    'GUU': 'V', 'GUC': 'V',
    'GUA': 'V', 'GUG': 'V',
    #Alanine
    'GCU': 'A', 'GCC': 'A',
    'GCA': 'A', 'GCG': 'A',
    #Aspartate
    'GAU': 'D', 'GAC': 'D',
    #Glutamate
```

```
    'GAA': 'E', 'GAG': 'E',
    #Glycine
    'GGU': 'G', 'GGC': 'G',
    'GGA': 'G', 'GGG': 'G',
    #Stop codon
    'UAA': '*', 'UAG': '*', 'UGA': '*',
}

#input sequence
def validate_sequence(sequence):
    if 'T' in sequence and 'U' in sequence:
        return None, "Invalid: sequence cannot contain both T and U."
    elif 'T' in sequence:
        valid = all(c in 'ATCG' for c in sequence)
        return ('DNA', None) if valid else (None, "Invalid DNA: only A, T,
C, G are allowed.")
    elif 'U' in sequence:
        valid = all(c in 'AUCG' for c in sequence)
        return ('RNA', None) if valid else (None, "Invalid RNA: only A, U,
C, G are allowed.")
    else:
        return ('DNA', None)

#Convert DNA to RNA
def convert_dna_to_rna(dna_sequence):
    return dna_sequence.replace('T', 'U')

#Check start and stop codons
def check_start_stop_codons(rna_sequence):
    start_index = rna_sequence.find('AUG')
    if start_index == -1:
        return None,


    coding_region = rna_sequence[start_index:]
    found_stop = False
    for i in range(0, len(coding_region) - 2, 3):
        codon = coding_region[i:i+3]
        if CODON_TABLE.get(codon) == '*':
            found_stop = True
```

```
            break

    if not found_stop:
        return None, "Nostop codon"

    return start_index, None

#Translate RNA
def translate_rna(rna_sequence, start_index):
    coding_region = rna_sequence[start_index:]
    protein = []

    for i in range(0, len(coding_region) - 2, 3):
        codon = coding_region[i:i+3]
        if len(codon) < 3:
            break
        aa = CODON_TABLE.get(codon)
        if aa is None:
            return None, f"Unknown codon: '{codon}'."
        if aa == '*':
            break
        protein.append(aa)

    if not protein:
        return None, "No amino acids translated."

    return ''.join(protein), None

#input
sequence = input("Enter DNA or RNA sequence: ").strip().upper()

#Validate
seq_type, error = validate_sequence(sequence)
if error:
    print(f"Validation error: {error}")
else:
    print(f"Detected sequence type: {seq_type}")

    #Convert DNA to RNA
    if seq_type == 'DNA':
```

```
        rna_sequence = convert_dna_to_rna(sequence)
        print(f"Converted RNA sequence: {rna_sequence}\n")
    else:
        rna_sequence = sequence
        print(f"RNA sequence: {rna_sequence}\n")

    #Find AUG and verify stop codon
    start_index, codon_error = check_start_stop_codons(rna_sequence)
    if codon_error:
        print(f"Codon check failed: {codon_error}")
    else:
        #Translate
        protein, trans_error = translate_rna(rna_sequence, start_index)
        if trans_error:
            print(f"Translation error: {trans_error}")
        else:
            print(f"\n A.A sequence:\n{protein}")
```

Week 3:

```
CODON_TABLE = {

    #Phenylalanine

    'UUU': 'F', 'UUC': 'F',

    #Leucine

    'UUA': 'L', 'UUG': 'L',

    'CUU': 'L', 'CUC': 'L',

    'CUA': 'L', 'CUG': 'L',

    #Serine

    'UCU': 'S', 'UCC': 'S',

    'UCA': 'S', 'UCG': 'S',

    'AGU': 'S', 'AGC': 'S',
```

```
    #Tyrosine
    'UAU': 'Y', 'UAC': 'Y',
    #Cysteine
    'UGU': 'C', 'UGC': 'C',
    #Tryptophan
    'UGG': 'W',
    #Proline
    'CCU': 'P', 'CCC': 'P',
    'CCA': 'P', 'CCG': 'P',
    #Histidine
    'CAU': 'H', 'CAC': 'H',
    #Glutamine
    'CAA': 'Q', 'CAG': 'Q',
    #Arginine
    'CGU': 'R', 'CGC': 'R',
    'CGA': 'R', 'CGG': 'R',
    'AGA': 'R', 'AGG': 'R',
    #Isoleucine
    'AUU': 'I', 'AUC': 'I',
    'AUA': 'I',
    #Methionine
    'AUG': 'M',
    #Threonine
```

```
    'ACU': 'T', 'ACC': 'T',
    'ACA': 'T', 'ACG': 'T',
    #Asparagine
    'AAU': 'N', 'AAC': 'N',
    #Lysine
    'AAA': 'K', 'AAG': 'K',
    #Valine
    'GUU': 'V', 'GUC': 'V',
    'GUA': 'V', 'GUG': 'V',
    #Alanine
    'GCU': 'A', 'GCC': 'A',
    'GCA': 'A', 'GCG': 'A',
    #Aspartate
    'GAU': 'D', 'GAC': 'D',
    #Glutamate
    'GAA': 'E', 'GAG': 'E',
    #Glycine
    'GGU': 'G', 'GGC': 'G',
    'GGA': 'G', 'GGG': 'G',
    #Stop codon
    'UAA': '*', 'UAG': '*', 'UGA': '*',
}
```

```
# —— WEEK 1 & 2
_____________________________________________________________________


#Validate input sequence

def validate_sequence(sequence):

    if 'T' in sequence and 'U' in sequence:

        return None, "Invalid: sequence cannot contain both T and U."

    elif 'T' in sequence:

        valid = all(c in 'ATCG' for c in sequence)

        return ('DNA', None) if valid else (None, "Invalid DNA: only A, T,
C, G are allowed.")

    elif 'U' in sequence:

        valid = all(c in 'AUCG' for c in sequence)

        return ('RNA', None) if valid else (None, "Invalid RNA: only A, U,
C, G are allowed.")

    else:

        return ('DNA', None)


#Convert DNA to RNA

def convert_dna_to_rna(dna_sequence):

    return dna_sequence.replace('T', 'U')


#Check start and stop codons

def check_start_stop_codons(rna_sequence):
```

```
    start_index = rna_sequence.find('AUG')

    if start_index == -1:

        return None, "No start codon (AUG) found in sequence."


    coding_region = rna_sequence[start_index:]

    found_stop = False

    for i in range(0, len(coding_region) - 2, 3):

        codon = coding_region[i:i+3]

        if CODON_TABLE.get(codon) == '*':

            found_stop = True

            break


    if not found_stop:

        return None, "No in-frame stop codon found."


    return start_index, None


#Translate RNA

def translate_rna(rna_sequence, start_index):

    coding_region = rna_sequence[start_index:]

    protein = []


    for i in range(0, len(coding_region) - 2, 3):
```

```
        codon = coding_region[i:i+3]
        if len(codon) < 3:
            break
        aa = CODON_TABLE.get(codon)
        if aa is None:
            return None, f"Unknown codon: '{codon}'."
        if aa == '*':
            break
        protein.append(aa)

    if not protein:
        return None, "No amino acids translated."

    return ''.join(protein), None

#Input
sequence = input("Enter DNA or RNA sequence: ").strip().upper()

#Validate
seq_type, error = validate_sequence(sequence)
if error:
    print(f"Validation error: {error}")
else:
```

```
    print(f"Detected sequence type: {seq_type}")


    #Convert DNA to RNA
    if seq_type == 'DNA':
        rna_sequence = convert_dna_to_rna(sequence)
        print(f"Converted RNA sequence: {rna_sequence}\n")
    else:
        rna_sequence = sequence
        print(f"RNA sequence: {rna_sequence}\n")


    #Find AUG and verify stop codon
    start_index, codon_error = check_start_stop_codons(rna_sequence)
    if codon_error:
        print(f"Codon check failed: {codon_error}")
    else:
        #Translate
        protein, trans_error = translate_rna(rna_sequence, start_index)
        if trans_error:
            print(f"Translation error: {trans_error}")
        else:
            print(f"\nA.A sequence:\n{protein}")
```

```
#
==========================================================================
===

# WEEK 3: Reading Frames

#
==========================================================================
===



#Week 3: Reverse complement of a DNA sequence (for reverse strand frames)

def reverse_complement(dna_sequence):

    complement = {'A': 'T', 'T': 'A', 'C': 'G', 'G': 'C'}

    return ''.join(complement[base] for base in reversed(dna_sequence))



#Week 3: Generate all 6 reading frames (3 forward then 3 reverse)

def get_all_reading_frames(dna_sequence):

    rev_comp = reverse_complement(dna_sequence)

    frames = {}

    for offset in range(3):

        frames[f"5'3' Frame {offset+1}"] =
convert_dna_to_rna(dna_sequence[offset:])

    for offset in range(3):

        frames[f"3'5' Frame {offset+1}"] =
convert_dna_to_rna(rev_comp[offset:])

    return frames



#Week 3: Translate from first AUG to first in-frame stop codon only
```

```
def translate_frame_orf(rna_sequence):

    #Find first start codon

    start_index = rna_sequence.find('AUG')

    if start_index == -1:

        return None, None, None, "No start codon (AUG) found."



    #Translate from AUG until first in-frame stop codon

    coding_region = rna_sequence[start_index:]

    protein = []

    stop_index = None



    for i in range(0, len(coding_region) - 2, 3):

        codon = coding_region[i:i+3]

        if len(codon) < 3:

            break

        aa = CODON_TABLE.get(codon)

        if aa == '*':

            stop_index = start_index + i  #position of stop codon in full
frame

            break

        if aa is None:

            protein.append('?')

        else:

            protein.append(aa)
```

```
    if stop_index is None:
        return None, None, None, "No in-frame stop codon found after AUG."

    return start_index, stop_index, ''.join(protein), None

#Week 3: Process and display one reading frame
def process_frame(frame_name, rna_sequence):
    print(f"\n{frame_name}")
    print(f"Frame Sequence : {rna_sequence}")

    start_index, stop_index, protein, error =
translate_frame_orf(rna_sequence)

    if error:
        print(f"Translation     : {error}")
    else:
        print(f"Start codon (AUG) found at position : {start_index}")
        print(f"Stop codon found at position        : {stop_index}")
        print(f"Translation                         : {protein}")

#Week 3: Run all reading frames on the input sequence
if 'seq_type' in dir() and not error:
    if seq_type == 'DNA':
```

```
        print("\n All 6 Reading Frames")

        frames = get_all_reading_frames(sequence)

        for frame_name, frame_rna in frames.items():

            process_frame(frame_name, frame_rna)

    else:

        print("\n 3 Forward Reading Frames (RNA)")

        for offset in range(3):

            process_frame(f"5'3' Frame {offset+1}", rna_sequence[offset:])
```

```
Enter DNA or RNA sequence:
AGCCCTCCAGGACAGGCTGCATCAGAAGAGGCCATCAAGCAGATCACTGTCCTTCTGCCATGGCCCTGTG
GATGCGCCTCCTGCCCCTGCTGGCGCTGCTGGCCCTCTGGGGACCTGACCCAGCCGCAGCCTTTGTGAAC
CAACACCTGTGCGGCTCACACCTGGTGGAAGCTCTCTACCTAGTGTGCGGGGAACGAGGCTTCTTCTACA
CACCCAAGACCCGCCGGGAGGCAGAGGACCTGCAGGTGGGGCAGGTGGAGCTGGGCGGGGGCCCTGGTGC
AGGCAGCCTGCAGCCCTTGGCCCTGGAGGGGTCCCTGCAGAAGCGTGGCATTGTGGAACAATGCTGTACC
AGCATCTGCTCCCTCTACCAGCTGGAGAACTACTGCAACTAGACGCAGCCCGCAGGCAGCCCCACACCCG
CCGCCTCCTGCACCGAGAGAGATGGAATAAAGCCCTTGAACCAGC
Detected sequence type: DNA
Converted RNA sequence:
AGCCCUCCAGGACAGGCUGCAUCAGAAGAGGCCAUCAAGCAGAUCACUGUCCUUCUGCCAUGGCCCUGUG
GAUGCGCCUCCUGCCCCUGCUGGCGCUGCUGGCCCUCUGGGGACCUGACCCAGCCGCAGCCUUUGUGAAC
CAACACCUGUGCGGCUCACACCUGGUGGAAGCUCUCUACCUAGUGUGCGGGGAACGAGGCUUCUUCUACA
CACCCAAGACCCGCCGGGAGGCAGAGGACCUGCAGGUGGGGCAGGUGGAGCUGGGCGGGGGCCCUGGUGC
AGGCAGCCUGCAGCCCUUGGCCCUGGAGGGGUCCCUGCAGAAGCGUGGCAUUGUGGAACAAUGCUGUACC
AGCAUCUGCUCCCUCUACCAGCUGGAGAACUACUGCAACUAGACGCAGCCCGCAGGCAGCCCCACACCCG
CCGCCUCCUGCACCGAGAGAGAUGGAAUAAAGCCCUUGAACCAGC
```

```
A.A sequence:
MALWMRLLPLLALLALWGPDPAAAFVNQHLCGSHLVEALYLVCGERGFFYTPKTRREAEDLQVGQVELGG
GPGAGSLQPLALEGSLQKRGIVEQCCTSICSLYQLENYCN
```

All 6 Reading Frames

5'3' Frame 1
Frame Sequence :
AGCCCUCCAGGACAGGCUGCAUCAGAAGAGGCCAUCAAGCAGAUCACUGUCCUUCUGCCAUGGCCCUGUG
GAUGCGCCUCCUGCCCCUGCUGGCGCUGCUGGCCCUCUGGGGACCUGACCCAGCCGCAGCCUUUGUGAAC
CAACACCUGUGCGGCUCACACCUGGUGGAAGCUCUCUACCUAGUGUGCGGGGAACGAGGCUUCUUCUACA
CACCCAAGACCCGCCGGGAGGCAGAGGACCUGCAGGUGGGGCAGGUGGAGCUGGGCGGGGGCCCUGGUGC
AGGCAGCCUGCAGCCCUUGGCCCUGGAGGGGUCCCUGCAGAAGCGUGGCAUUGUGGAACAAUGCUGUACC
AGCAUCUGCUCCCUCUACCAGCUGGAGAACUACUGCAACUAGACGCAGCCCGCAGGCAGCCCCACACCCG
CCGCCUCCUGCACCGAGAGAGAUGGAAUAAAGCCCUUGAACCAGC
Start codon (AUG) found at position : 59
Stop codon found at position : 389
Translation :
MALWMRLLPLLALLALWGPDPAAAFVNQHLCGSHLVEALYLVCGERGFFYTPKTRREAEDLQVGQVELGG
GPGAGSLQPLALEGSLQKRGIVEQCCTSICSLYQLENYCN

5'3' Frame 2
Frame Sequence :
GCCCUCCAGGACAGGCUGCAUCAGAAGAGGCCAUCAAGCAGAUCACUGUCCUUCUGCCAUGGCCCUGUGG
AUGCGCCUCCUGCCCCUGCUGGCGCUGCUGGCCCUCUGGGGACCUGACCCAGCCGCAGCCUUUGUGAACC
AACACCUGUGCGGCUCACACCUGGUGGAAGCUCUCUACCUAGUGUGCGGGGAACGAGGCUUCUUCUACAC
ACCCAAGACCCGCCGGGAGGCAGAGGACCUGCAGGUGGGGCAGGUGGAGCUGGGCGGGGGCCCUGGUGCA
GGCAGCCUGCAGCCCUUGGCCCUGGAGGGGUCCCUGCAGAAGCGUGGCAUUGUGGAACAAUGCUGUACCA
GCAUCUGCUCCCUCUACCAGCUGGAGAACUACUGCAACUAGACGCAGCCCGCAGGCAGCCCCACACCCGC
CGCCUCCUGCACCGAGAGAGAUGGAAUAAAGCCCUUGAACCAGC
Start codon (AUG) found at position : 58
Stop codon found at position : 388
Translation :
MALWMRLLPLLALLALWGPDPAAAFVNQHLCGSHLVEALYLVCGERGFFYTPKTRREAEDLQVGQVELGG
GPGAGSLQPLALEGSLQKRGIVEQCCTSICSLYQLENYCN

5'3' Frame 3
Frame Sequence :
CCCUCCAGGACAGGCUGCAUCAGAAGAGGCCAUCAAGCAGAUCACUGUCCUUCUGCCAUGGCCCUGUGGA
UGCGCCUCCUGCCCCUGCUGGCGCUGCUGGCCCUCUGGGGACCUGACCCAGCCGCAGCCUUUGUGAACCA
ACACCUGUGCGGCUCACACCUGGUGGAAGCUCUCUACCUAGUGUGCGGGGAACGAGGCUUCUUCUACACA
CCCAAGACCCGCCGGGAGGCAGAGGACCUGCAGGUGGGGCAGGUGGAGCUGGGCGGGGGCCCUGGUGCAG

GCAGCCUGCAGCCCUUGGCCCUGGAGGGGUCCCUGCAGAAGCGUGGCAUUGUGGAACAAUGCUGUACCAG
CAUCUGCUCCCUCUACCAGCUGGAGAACUACUGCAACUAGACGCAGCCCGCAGGCAGCCCCACACCCGCC
GCCUCCUGCACCGAGAGAGAUGGAAUAAAGCCCUUGAACCAGC
Start codon (AUG) found at position : 57
Stop codon found at position : 387
Translation :
MALWMRLLPLLALLALWGPDPAAAFVNQHLCGSHLVEALYLVCGERGFFYTPKTRREAEDLQVGQVELGG
GPGAGSLQPLALEGSLQKRGIVEQCCTSICSLYQLENYCN

3'5' Frame 1
Frame Sequence :
GCUGGUUCAAGGGCUUUAUUCCAUCUCUCUCGGUGCAGGAGGCGGCGGGUGUGGGGCUGCCUGCGGGCUG
CGUCUAGUUGCAGUAGUUCUCCAGCUGGUAGAGGGAGCAGAUGCUGGUACAGCAUUGUUCCACAAUGCCA
CGCUUCUGCAGGGACCCCUCCAGGGCCAAGGGCUGCAGGCUGCCUGCACCAGGGCCCCCGCCCAGCUCCA
CCUGCCCCACCUGCAGGUCCUCUGCCUCCCGGCGGGUCUUGGGUGUGUAGAAGAAGCCUCGUUCCCCGCA
CACUAGGUAGAGAGCUUCCACCAGGUGUGAGCCGCACAGGUGUUGGUUCACAAAGGCUGCGGCUGGGUCA
GGUCCCCAGAGGGCCAGCAGCGCCAGCAGGGGCAGGAGGCGCAUCCACAGGGCCAUGGCAGAAGGACAGU
GAUCUGCUUGAUGGCCUCUUCUGAUGCAGCCUGUCCUGGAGGGCU
Start codon (AUG) found at position : 110
Stop codon found at position : 257
Translation :
MLVQHCSTMPRFCRDPSRAKGCRLPAPGPPPSSTCPTCRSSASRRVLGV

3'5' Frame 2
Frame Sequence :
CUGGUUCAAGGGCUUUAUUCCAUCUCUCUCGGUGCAGGAGGCGGCGGGUGUGGGGCUGCCUGCGGGCUGC
GUCUAGUUGCAGUAGUUCUCCAGCUGGUAGAGGGAGCAGAUGCUGGUACAGCAUUGUUCCACAAUGCCAC
GCUUCUGCAGGGACCCCUCCAGGGCCAAGGGCUGCAGGCUGCCUGCACCAGGGCCCCCGCCCAGCUCCAC
CUGCCCCACCUGCAGGUCCUCUGCCUCCCGGCGGGUCUUGGGUGUGUAGAAGAAGCCUCGUUCCCCGCAC
ACUAGGUAGAGAGCUUCCACCAGGUGUGAGCCGCACAGGUGUUGGUUCACAAAGGCUGCGGCUGGGUCAG
GUCCCCAGAGGGCCAGCAGCGCCAGCAGGGGCAGGAGGCGCAUCCACAGGGCCAUGGCAGAAGGACAGUG
AUCUGCUUGAUGGCCUCUUCUGAUGCAGCCUGUCCUGGAGGGCU
Start codon (AUG) found at position : 109
Stop codon found at position : 256
Translation :
MLVQHCSTMPRFCRDPSRAKGCRLPAPGPPPSSTCPTCRSSASRRVLGV

3'5' Frame 3

```
Frame Sequence :
UGGUUCAAGGGCUUUAUUCCAUCUCUCUCGGUGCAGGAGGCGGCGGGUGUGGGGCUGCCUGCGGGCUGCG
UCUAGUUGCAGUAGUUCUCCAGCUGGUAGAGGGAGCAGAUGCUGGUACAGCAUUGUUCCACAAUGCCACG
CUUCUGCAGGGACCCCUCCAGGGCCAAGGGCUGCAGGCUGCCUGCACCAGGGCCCCCGCCCAGCUCCACC
UGCCCCACCUGCAGGUCCUCUGCCUCCCGGCGGGUCUUGGGUGUGUAGAAGAAGCCUCGUUCCCCGCACA
CUAGGUAGAGAGCUUCCACCAGGUGUGAGCCGCACAGGUGUUGGUUCACAAAGGCUGCGGCUGGGUCAGG
UCCCCAGAGGGCCAGCAGCGCCAGCAGGGGCAGGAGGCGCAUCCACAGGGCCAUGGCAGAAGGACAGUGA
UCUGCUUGAUGGCCUCUUCUGAUGCAGCCUGUCCUGGAGGGCU
Start codon (AUG) found at position : 108
Stop codon found at position        : 255
Translation                         :
MLVQHCSTMPRFCRDPSRAKGCRLPAPGPPPSSTCPTCRSSASRRVLGV
```

Week 4:

Tkinter and file handling.
Code:

```
import tkinter as tk
from tkinter import ttk, filedialog, messagebox
from Bio import SeqIO
import os

# CODON TABLE
CODON_TABLE = {
    'UUU': 'F', 'UUC': 'F',
    'UUA': 'L', 'UUG': 'L',
    'CUU': 'L', 'CUC': 'L', 'CUA': 'L', 'CUG': 'L',
    'UCU': 'S', 'UCC': 'S', 'UCA': 'S', 'UCG': 'S',
    'AGU': 'S', 'AGC': 'S',
    'UAU': 'Y', 'UAC': 'Y',
    'UGU': 'C', 'UGC': 'C',
    'UGG': 'W',
    'CCU': 'P', 'CCC': 'P', 'CCA': 'P', 'CCG': 'P',
    'CAU': 'H', 'CAC': 'H',
    'CAA': 'Q', 'CAG': 'Q',
    'CGU': 'R', 'CGC': 'R', 'CGA': 'R', 'CGG': 'R',
    'AGA': 'R', 'AGG': 'R',
```

```
    'AUU': 'I', 'AUC': 'I', 'AUA': 'I',
    'AUG': 'M',
    'ACU': 'T', 'ACC': 'T', 'ACA': 'T', 'ACG': 'T',
    'AAU': 'N', 'AAC': 'N',
    'AAA': 'K', 'AAG': 'K',
    'GUU': 'V', 'GUC': 'V', 'GUA': 'V', 'GUG': 'V',
    'GCU': 'A', 'GCC': 'A', 'GCA': 'A', 'GCG': 'A',
    'GAU': 'D', 'GAC': 'D',
    'GAA': 'E', 'GAG': 'E',
    'GGU': 'G', 'GGC': 'G', 'GGA': 'G', 'GGG': 'G',
    'UAA': '*', 'UAG': '*', 'UGA': '*',
}

# LOGIC (Week 1, 2, 3)

def validate_sequence(sequence):
    if 'T' in sequence and 'U' in sequence:
        return None, "Invalid: sequence cannot contain both T and U."
    elif 'T' in sequence:
        valid = all(c in 'ATCG' for c in sequence)
        return ('DNA', None) if valid else (None, "Invalid DNA: only A, T,
C, G allowed.")
    elif 'U' in sequence:
        valid = all(c in 'AUCG' for c in sequence)
        return ('RNA', None) if valid else (None, "Invalid RNA: only A, U,
C, G allowed.")
    else:
        return ('DNA', None)

def convert_dna_to_rna(dna_sequence):
    return dna_sequence.replace('T', 'U')

def check_start_stop_codons(rna_sequence):
    start_index = rna_sequence.find('AUG')
    if start_index == -1:
        return None, "No start codon (AUG) found in sequence."
    coding_region = rna_sequence[start_index:]
    found_stop = False
    for i in range(0, len(coding_region) - 2, 3):
        codon = coding_region[i:i+3]
```

```
        if CODON_TABLE.get(codon) == '*':
            found_stop = True
            break
    if not found_stop:
        return None, "No in-frame stop codon found."
    return start_index, None

def translate_rna(rna_sequence, start_index):
    coding_region = rna_sequence[start_index:]
    protein = []
    for i in range(0, len(coding_region) - 2, 3):
        codon = coding_region[i:i+3]
        if len(codon) < 3:
            break
        aa = CODON_TABLE.get(codon)
        if aa is None:
            return None, f"Unknown codon: '{codon}'."
        if aa == '*':
            break
        protein.append(aa)
    if not protein:
        return None, "No amino acids translated."
    return ''.join(protein), None

def reverse_complement(dna_sequence):
    complement = {'A': 'T', 'T': 'A', 'C': 'G', 'G': 'C'}
    return ''.join(complement[base] for base in reversed(dna_sequence))

def get_all_reading_frames(dna_sequence):
    rev_comp = reverse_complement(dna_sequence)
    frames = {}
    for offset in range(3):
        frames[f"5'3' Frame {offset+1}"] =
convert_dna_to_rna(dna_sequence[offset:])
    for offset in range(3):
        frames[f"3'5' Frame {offset+1}"] =
convert_dna_to_rna(rev_comp[offset:])
    return frames

def translate_frame_orf(rna_sequence):
```

```
    start_index = rna_sequence.find('AUG')
    if start_index == -1:
        return None, None, None, "No start codon (AUG) found."
    coding_region = rna_sequence[start_index:]
    protein = []
    stop_index = None
    for i in range(0, len(coding_region) - 2, 3):
        codon = coding_region[i:i+3]
        if len(codon) < 3:
            break
        aa = CODON_TABLE.get(codon)
        if aa == '*':
            stop_index = start_index + i
            break
        if aa is None:
            protein.append('?')
        else:
            protein.append(aa)
    if stop_index is None:
        return None, None, None, "No in-frame stop codon found after AUG."
    return start_index, stop_index, ''.join(protein), None

# FILE READING

def read_fasta(filepath):
    records = list(SeqIO.parse(filepath, "fasta"))
    if not records:
        return None, None, "No sequences found in FASTA file."
    record = records[0]
    return str(record.seq).upper(), record.id, None

def read_text_file(filepath):
    with open(filepath, 'r') as f:
        sequence = f.read().strip().upper()
    return sequence, os.path.basename(filepath), None

# THEME COLOURS
BG          = "#0d1117"
SURFACE     = "#161b22"
SURFACE2    = "#1c2128"
```

```
BORDER      = "#30363d"
ACCENT      = "#58a6ff"
ACCENT2     = "#3fb950"
STOP_COLOR  = "#f85149"
MUTED       = "#8b949e"
TEXT        = "#e6edf3"
TEXT_DIM    = "#c9d1d9"
FONT_MONO   = ("Courier New", 10)
FONT_LABEL  = ("Courier New", 10, "bold")
FONT_TITLE  = ("Courier New", 14, "bold")
FONT_SMALL  = ("Courier New", 9)

# MAIN GUI CLASS

class TranslatorApp:
    def __init__(self, root):
        self.root = root
        self.root.title("Translation Tool")
        self.root.configure(bg=BG)
        self.root.geometry("1050x780")
        self.root.resizable(True, True)
        self._build_ui()

    # Build full UI
    def _build_ui(self):
        # Title bar
        title_frame = tk.Frame(self.root, bg=BG, pady=16)
        title_frame.pack(fill='x', padx=24)

        tk.Label(title_frame, text="⬡  TRANSLATION TOOL",
                 font=FONT_TITLE, fg=ACCENT, bg=BG).pack(side='left')

        # Separator
        tk.Frame(self.root, bg=BORDER, height=1).pack(fill='x', padx=24)

        # Main layout — left panel + right output
        main = tk.Frame(self.root, bg=BG)
        main.pack(fill='both', expand=True, padx=24, pady=16)

        self._build_left_panel(main)
```

```
        self._build_right_panel(main)

    # Left panel: input controls
    def _build_left_panel(self, parent):
        left = tk.Frame(parent, bg=SURFACE, bd=0,
                        highlightbackground=BORDER, highlightthickness=1)
        left.pack(side='left', fill='y', ipadx=12, ipady=12, padx=(0, 12))

        # Section: sequence input
        self._section_label(left, "SEQUENCE INPUT")

        self.seq_text = tk.Text(left, height=8, width=38,
                                font=FONT_MONO, fg=TEXT, bg=SURFACE2,
                                insertbackground=ACCENT, relief='flat',
                                highlightbackground=BORDER,
highlightthickness=1,
                                wrap='word')
        self.seq_text.pack(padx=12, pady=(0, 8))

        # File buttons
        file_row = tk.Frame(left, bg=SURFACE)
        file_row.pack(fill='x', padx=12, pady=(0, 12))

        self._btn(file_row, "Load FASTA", self._load_fasta,
                  ACCENT, BG).pack(side='left', padx=(0, 6))
        self._btn(file_row, "Load .txt", self._load_txt,
                  MUTED, BG).pack(side='left')

        # Clear button
        self._btn(left, "Clear Input", self._clear_input,
                  SURFACE2, MUTED, full=True).pack(padx=12, pady=(0, 12),
fill='x')

        # Separator
        tk.Frame(left, bg=BORDER, height=1).pack(fill='x', padx=12,
pady=8)

        # Section: options
        self._section_label(left, "OPTIONS")
```

```
        # Show reading frames toggle
        self.show_frames = tk.BooleanVar(value=True)
        chk = tk.Checkbutton(left, text="Show all reading frames",
                             variable=self.show_frames,
                             font=FONT_SMALL, fg=TEXT_DIM, bg=SURFACE,
                             selectcolor=SURFACE2,
activebackground=SURFACE,
                             activeforeground=TEXT)
        chk.pack(anchor='w', padx=12, pady=4)

        # Separator
        tk.Frame(left, bg=BORDER, height=1).pack(fill='x', padx=12,
pady=8)

        # Translate button
        self._btn(left, "▶  TRANSLATE", self._run_translation,
                  ACCENT2, BG, full=True, padx=12, pady=10,
                  font=("Courier New", 11, "bold")).pack(padx=12, pady=(0,
8), fill='x')

        # Status label
        self.status_var = tk.StringVar(value="Ready.")
        tk.Label(left, textvariable=self.status_var,
                 font=FONT_SMALL, fg=MUTED, bg=SURFACE,
                 wraplength=250, justify='left').pack(padx=12, anchor='w')

    # Right panel: output
    def _build_right_panel(self, parent):
        right = tk.Frame(parent, bg=BG)
        right.pack(side='left', fill='both', expand=True)

        # Tabs
        style = ttk.Style()
        style.theme_use('default')
        style.configure('TNotebook',         background=BG,
borderwidth=0)
        style.configure('TNotebook.Tab',     background=SURFACE,
foreground=MUTED,
                        font=FONT_SMALL, padding=[12, 6])
        style.map('TNotebook.Tab',
```

```
                 background=[('selected', SURFACE2)],
                 foreground=[('selected', ACCENT)])

        self.notebook = ttk.Notebook(right)
        self.notebook.pack(fill='both', expand=True)

        # Tab 1: Main translation
        tab1 = tk.Frame(self.notebook, bg=BG)
        self.notebook.add(tab1, text='  Translation  ')
        self.main_output = self._output_box(tab1)

        # Tab 2: Reading frames
        tab2 = tk.Frame(self.notebook, bg=BG)
        self.notebook.add(tab2, text='  Reading Frames  ')
        self.frames_output = self._output_box(tab2)

        # Tab 3: Sequence info
        tab3 = tk.Frame(self.notebook, bg=BG)
        self.notebook.add(tab3, text='  Sequence Info  ')
        self.info_output = self._output_box(tab3)

    # Helpers
    def _section_label(self, parent, text):
        tk.Label(parent, text=text, font=("Courier New", 8, "bold"),
                 fg=MUTED, bg=SURFACE).pack(anchor='w', padx=12, pady=(12,
4))

    def _btn(self, parent, text, cmd, bg, fg,
             full=False, padx=0, pady=6, font=FONT_LABEL):
        return tk.Button(parent, text=text, command=cmd,
                         font=font, fg=fg, bg=bg,
                         relief='flat', cursor='hand2',
                         activebackground=BORDER, activeforeground=TEXT,
                         padx=padx, pady=pady)

    def _output_box(self, parent):
        frame = tk.Frame(parent, bg=BG)
        frame.pack(fill='both', expand=True, padx=8, pady=8)

        box = tk.Text(frame, font=FONT_MONO, fg=TEXT, bg=SURFACE,
```

```
                        relief='flat', wrap='word', state='disabled',
                        highlightbackground=BORDER, highlightthickness=1,
                        spacing3=2)

        scroll = tk.Scrollbar(frame, command=box.yview, bg=SURFACE,
                              troughcolor=BG, relief='flat')
        box.configure(yscrollcommand=scroll.set)

        scroll.pack(side='right', fill='y')
        box.pack(side='left', fill='both', expand=True)

        # Text tags for coloring
        box.tag_configure('header',  foreground=ACCENT,
font=("Courier New", 10, "bold"))
        box.tag_configure('success', foreground=ACCENT2)
        box.tag_configure('error',   foreground=STOP_COLOR)
        box.tag_configure('muted',   foreground=MUTED)
        box.tag_configure('label',   foreground=TEXT_DIM,
font=("Courier New", 10, "bold"))
        box.tag_configure('protein', foreground="#ffa657",
font=("Courier New", 10, "bold"))
        box.tag_configure('stop',    foreground=STOP_COLOR)
        box.tag_configure('frame',   foreground="#d2a8ff")

        return box

    def _write(self, box, text, tag=None):
        box.configure(state='normal')
        if tag:
            box.insert('end', text, tag)
        else:
            box.insert('end', text)
        box.configure(state='disabled')

    def _clear_box(self, box):
        box.configure(state='normal')
        box.delete('1.0', 'end')
        box.configure(state='disabled')

    #File loading
```

```
    def _load_fasta(self):
        path = filedialog.askopenfilename(
            title="Open FASTA file",
            filetypes=[("FASTA files", "*.fasta *.fa *.fna"), ("All
files", "*.*")]
        )
        if not path:
            return
        seq, seq_id, err = read_fasta(path)
        if err:
            messagebox.showerror("Error", err)
            return
        self.seq_text.delete('1.0', 'end')
        self.seq_text.insert('end', seq)
        self.status_var.set(f"Loaded FASTA: {seq_id}")

    def _load_txt(self):
        path = filedialog.askopenfilename(
            title="Open text file",
            filetypes=[("Text files", "*.txt"), ("All files", "*.*")]
        )
        if not path:
            return
        seq, name, err = read_text_file(path)
        if err:
            messagebox.showerror("Error", err)
            return
        self.seq_text.delete('1.0', 'end')
        self.seq_text.insert('end', seq)
        self.status_var.set(f"Loaded file: {name}")

    def _clear_input(self):
        self.seq_text.delete('1.0', 'end')
        self._clear_box(self.main_output)
        self._clear_box(self.frames_output)
        self._clear_box(self.info_output)
        self.status_var.set("Ready.")

    # Main translation logic
    def _run_translation(self):
```

```
        # Strip newlines, spaces and carriage returns — handles pasted
sequences
        raw = self.seq_text.get('1.0', 'end')
        sequence = ''.join(raw.split()).upper()
        if not sequence:
            messagebox.showwarning("No Input", "Please enter or load a
sequence first.")
            return

        self._clear_box(self.main_output)
        self._clear_box(self.frames_output)
        self._clear_box(self.info_output)

        out  = self.main_output
        fout = self.frames_output
        iout = self.info_output

        # Validate
        seq_type, error = validate_sequence(sequence)
        if error:
            self._write(out, "VALIDATION ERROR\n", 'header')
            self._write(out, error + "\n", 'error')
            self.status_var.set("Validation failed.")
            return

        #Sequence info tab
        self._write(iout, "SEQUENCE INFORMATION\n", 'header')
        self._write(iout, "—" * 50 + "\n", 'muted')
        self._write(iout, f"Type        : ", 'label')
        self._write(iout, f"{seq_type}\n", 'success')
        self._write(iout, f"Length      : ", 'label')
        self._write(iout, f"{len(sequence)} bases\n")
        self._write(iout, f"GC Content  : ", 'label')
        gc = (sequence.count('G') + sequence.count('C')) / len(sequence) *
100
        self._write(iout, f"{gc:.2f}%\n")
        self._write(iout, f"\nRaw Sequence:\n", 'label')
        self._write(iout, sequence + "\n", 'muted')

        # Convert to RNA
```

```
        if seq_type == 'DNA':
            rna_sequence = convert_dna_to_rna(sequence)
        else:
            rna_sequence = sequence

        self._write(iout, f"\nRNA Sequence:\n", 'label')
        self._write(iout, rna_sequence + "\n", 'muted')

        #Main translation tab
        self._write(out, "TRANSLATION RESULT\n", 'header')
        self._write(out, "—" * 50 + "\n", 'muted')
        self._write(out, f"Sequence Type : ", 'label')
        self._write(out, f"{seq_type}\n", 'success')
        self._write(out, f"RNA Sequence  : ", 'label')
        self._write(out, f"{rna_sequence}\n\n", 'muted')

        start_index, codon_error = check_start_stop_codons(rna_sequence)
        if codon_error:
            self._write(out, "CODON CHECK FAILED\n", 'error')
            self._write(out, codon_error + "\n", 'error')
            self.status_var.set("Codon check failed.")
        else:
            self._write(out, f"Start codon (AUG) at position : ", 'label')
            self._write(out, f"{start_index}\n")

            protein, trans_error = translate_rna(rna_sequence,
start_index)
            if trans_error:
                self._write(out, "TRANSLATION ERROR\n", 'error')
                self._write(out, trans_error + "\n", 'error')
                self.status_var.set("Translation error.")
            else:
                self._write(out, f"\nAmino Acid Sequence:\n", 'label')
                self._write(out, protein + "\n", 'protein')
                self._write(out, f"\nLength : {len(protein)} amino
acids\n", 'muted')
                self.status_var.set(f"Done. {len(protein)} amino acids
translated.")

        #Reading frames tab
```

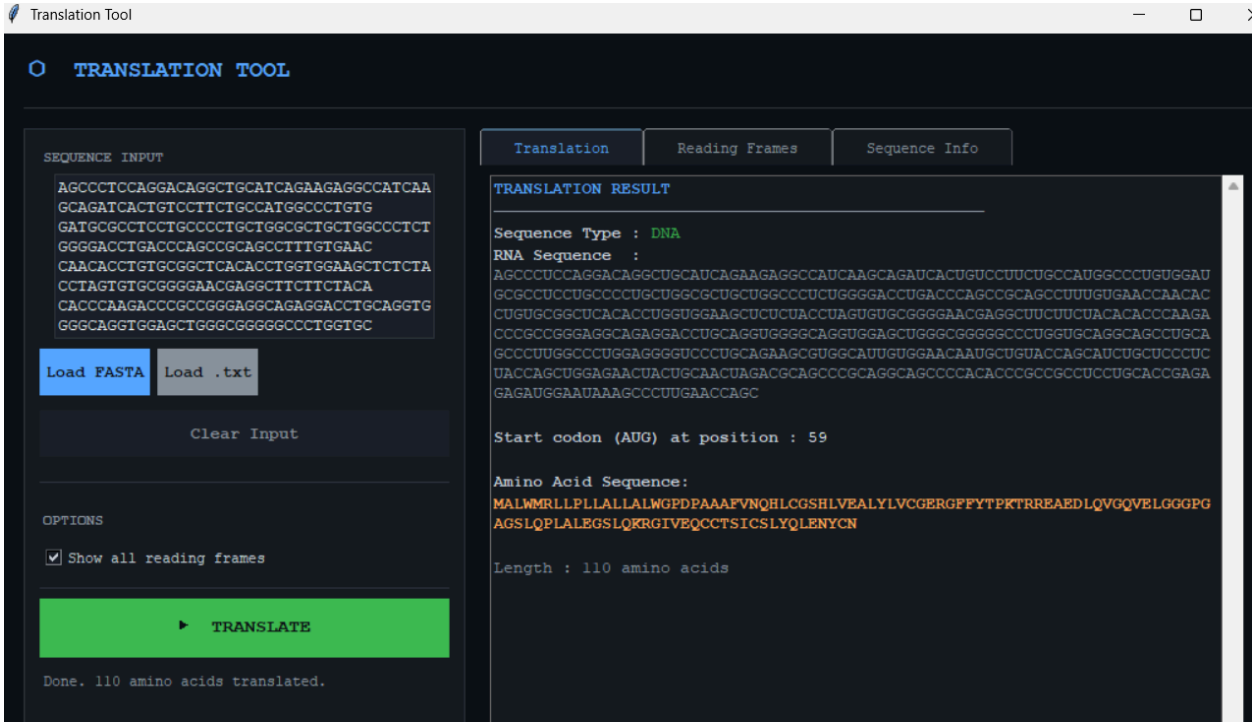
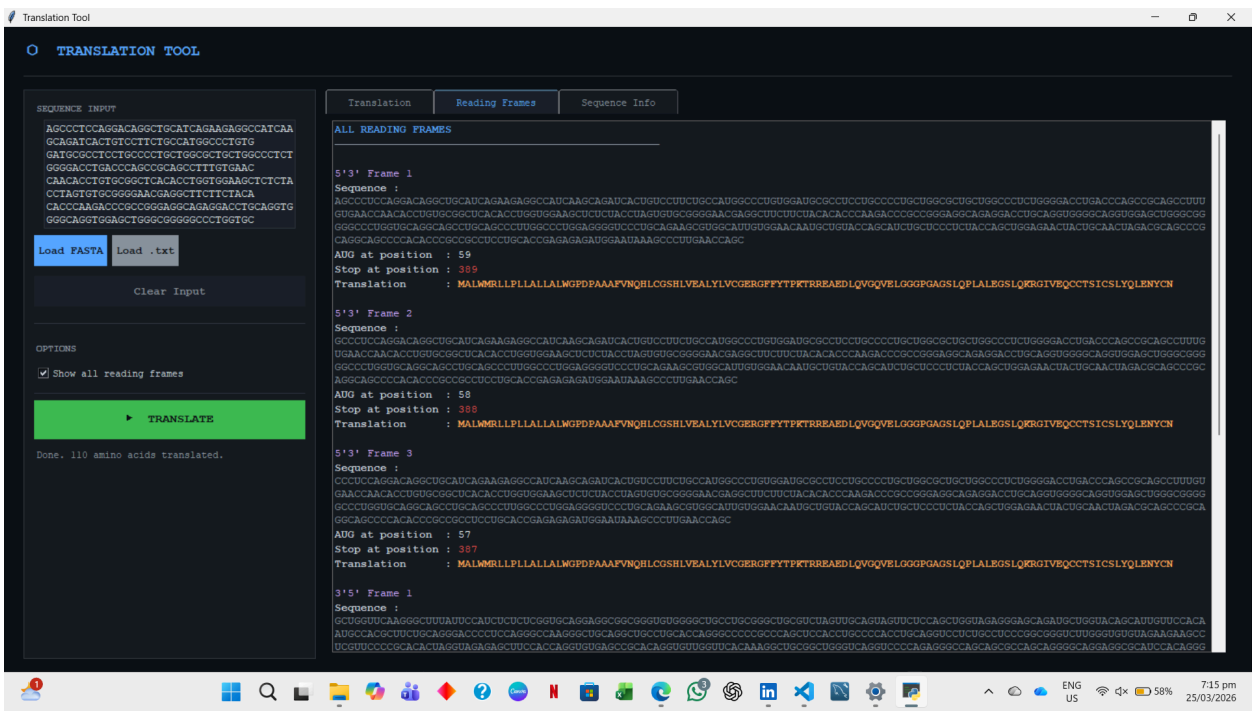
```
        if self.show_frames.get():
            self._write(fout, "ALL READING FRAMES\n", 'header')
            self._write(fout, "—" * 50 + "\n", 'muted')

            if seq_type == 'DNA':
                frames = get_all_reading_frames(sequence)
                for frame_name, frame_rna in frames.items():
                    self._write_frame(fout, frame_name, frame_rna)
            else:
                for offset in range(3):
                    self._write_frame(fout,
                                      f"5'3' Frame {offset+1}",
                                      rna_sequence[offset:])
        else:
            self._write(fout, "Reading frames disabled.\n", 'muted')
            self._write(fout, "Enable via the checkbox in the left
panel.\n", 'muted')

        # Switch to translation tab
        self.notebook.select(0)

    def _write_frame(self, box, frame_name, rna_sequence):
        self._write(box, f"\n{frame_name}\n", 'frame')
        self._write(box, f"Sequence : ", 'label')
        self._write(box, f"{rna_sequence}\n", 'muted')

        start_index, stop_index, protein, error =
translate_frame_orf(rna_sequence)

        if error:
            self._write(box, f"Result   : ", 'label')
            self._write(box, f"{error}\n", 'error')
        else:
            self._write(box, f"AUG at position  : ", 'label')
            self._write(box, f"{start_index}\n")
            self._write(box, f"Stop at position : ", 'label')
            self._write(box, f"{stop_index}\n", 'stop')
            self._write(box, f"Translation      : ", 'label')
            self._write(box, f"{protein}\n", 'protein')
```

```python
# RUN
root = tk.Tk()
app  = TranslatorApp(root)
root.mainloop()
```